Date

LECTURE-14

Monday
10/10/26

Rod Cutting: we are given a rod with length 'n' along with an array of prices which are of segments of rod. Kay ager rod iss length ka hova then ithe price.

Task: we can either sell rod without cutting in length of 5 as it is for price = 9

N = 5

Price of Segment	Segment Length
2	1
5	2
7	3
8	4
9	5

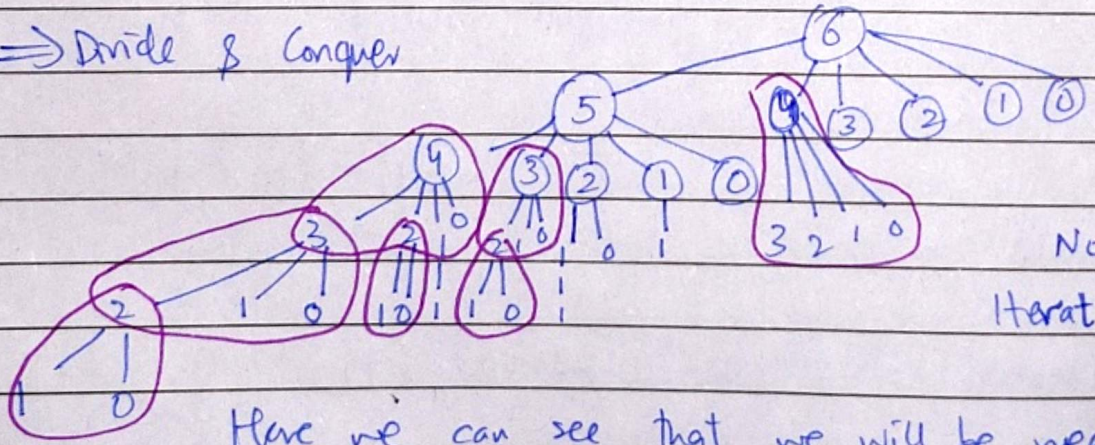
OR

we can try diff. cutting ratios to maximise the profit, Like:

2 : 3 = 5+7 = 12

12 > 9

⇒ Divide & Conquer



No. of Nodes to Iterate = $2^n - 1$

Here we can see that we will be needing branches again & again which will take complexity to 2^n if we'll compute each branch again & again.

Therefore use DP & store result of each branch's max in an array.

CamScanner

Date

★ We'll use a 'dp' array and each index will corresponds to the length of segment & the data of that index will be the maximum profit we can obtain from that length segment of rod.

• This is useful as we won't need to commute the max. profit again & again for i^{th} length of segment & we can simply take suppose max profit of 3rd length segment if we need to commute suppose 3+2.

General Case = Max (dp[i], price[j] + dp[i-j]) → Single cut

No cuts applied; price array say directly value uthao

lengths / index →	0	1	2	3	4	5
Price	0	3	5	6	7	8
DP	0	3	6	9	12	15

Index of DP array are N kay itne length kay suppos 5 length ka rod hoga tou max profit = 15

→ This could be "price[i]" too.

Explanation:

0 : rod length = 0 means 0 profit/price

1 : Length (i) = 1 ∴ no cuts could be done ∴ dp[1] = price[1] = 3

2 ; Length (i) = 2 could be either cut into:

- j=1 : i-j=1 = 3 + 3 = 6
- j=2 : i-j=0 (No cuts) = 5 (this is from price[2])

Max (6, 5) = 6 ∴ dp[2] = 6.

3: Length (i) = 3 could be either cut into: We won't directly find this; this will be found in recursion by pairs

a. 1 : 1 : 1 = 3+3+3 = 9

b. j=1 : i-j=2 = 3 + 6 = 9 → this '6' came from dp[2]. We did price[j] + dp[i-j]

c. j=3 : i-j=0 = 6 (this is price[3])

Max (9, 9) = 9 → Max (9, 6) = 9

∴ dp[i] = dp[3] = 9

NOTE: The result of a & b are compared with max first & then those are further compared with c. Thats why Max (dp[i], price + dp)

Pearl Notes

dp[2] ← Maximum profits used from dp array which shows why we don't need to commute max. again & again

dp[3]

Date

4. i = length = 4 could be cut into:

- 1:1:1:1 = 3+3+3+3 = 12
- 1:2:1 = 3+6+3 = 12
- 1:3 = 3+9 = 12
- 4 (no cut) = 7

Max (12, 12) = 12

Max (12, 12) = 12

Max (12, 7) = 12

~~Max (7, 12, 12, 12) = 12 ∴ dp[4] = 12~~

∴ dp[4] = 12

Code : Time = $O(N^2)$, Space = $O(N)$. (DP : Iterative)

```
rod cut ( price [], n)
    dp [0] = 0
    for i = 1 TO n                    // It's the current best/max we have while we are cutting the rods in diff segs like where each ratio is cumming max.
        dp [i] = 0

        for j = 1 TO i                // j=1, i=4 then 4-1 = remaining segment.
            dp [i] = MAX ( dp [i], price [j] + dp [i - j])
    return dp [n]                     // max profit for n length rod is in nth index of dp
```

Code : (Divide & Conquer) // I've a rod & need best price at length = 4 so I'll pass "(prices array, 4)" in func & will get "dp[4]" from func. i.e: the best price for length = 4.

```
rod cut ( p[n], n)
    if  n == 0
        return 0
    if  n == 1
        return p [1]
    else
        for i = 1 To n
            r_max = Max [r_max , p [j] + rod cut (p, n - i)]
    return r_max.
```

Pearl Notes

These are recursive calls

Date

N=4	N=3	N=2	N=1
P[1] + s(3) ①, 9	P[1] + s(2) 2, 6	P[1] + s(1) 3	P[1] 4
P[2] + s(2) 10	P[2] + s(1) 7	P[2] + s(0) 5	
P[3] + s(1) 11	P[1] + s(0) 8		
P[4] + s(0) 12			

Method:

1) N=4 requires P[1] + s(3). P[1] is fetched from array P.
   For s(3); go to N=3 at step 2
2) For N=3 which is need for N=4 here; we need P[1] + s(2).
   P[1] from array & s(2) from N=2.
3) For N=2 which is needed for N=3; we need P[1] + s(1).
   s(1) will be from N=1 P[1]
4) N=1 is base case & will return to "P[1] + s(1)" of "N=2".
   So branch 1 of N=2 is committed & we'll now see branch 2.
5) For branch 2 of N=2; P[2] from array & s(0), which is a base case that returns zero.
   Hence both branches of N=2 solved; so we'll save the max outta these two branches in DP array to use later. in DP[2]
6) return DP[2] to branch 1 of N=3.
7) Commute branch 2 of N=3 by DP[1] (already s(1) stored)
8) Commute branch 3 of N=3 by P[1].
   Now find max of N=3 and store in DP[3]
9) Return max from DP(3) to (Commute) branch 1 of N=4
10), 11), 12); Commute these using DP array and then find max from four branches of N=4 to store max in DP[4].

CamScanner